

1. Заголовок

Кейс «Проведение анализа защищенности приложения с помощью LLM моделей» — направлен на изучение темы информационной безопасности уровня Application, выработку подходов к определению защищенности информационной системы, автоматизацию действий на базе взаимодействия с LLM моделями.

Направление подготовки: 09.03.03 – Прикладная информатика (или аналогичное)

Образовательная программа: Прикладной искусственный интеллект

Семестр: 2-й семестр

2. Аннотация

Кейс «Проведение анализа защищенности приложения с помощью LLM моделей» представляет собой задачу по выработке подходов к подготовке подходов к определению уровня защищенности web приложения.

Учебная дисциплина: Проектный практикум / Искусственный интеллект.

Актуальность и практическая значимость:

Информационная безопасность является важным аспектом качества создаваемого ПО, который необходимо постоянно поддерживать через комплекс организационных и технических мер. В состав технических мер входит проведение автоматизированных проверок на наличие уязвимостей. Это сложный и комплексный процесс, требующий широкого круга знаний. Применение больших языковых моделей (LLM) в данной теме потенциально позволит снизить требуемый уровень знаний для пользователя, проводить проверку более верхнеуровнево, что повысит доступность данного типа проверки. Реализация этого кейса позволит студентам получить основные знания по проведению анализа защищенности и «упаковать» их в продукт – план тестирования для ИИ агента.

Форма выполнения: групповая.

Кейс направлен на формирование следующих ключевых компетенций:

Профессиональные компетенции (hard skills):

- **Алгоритмическое мышление:** Понимание и реализация вероятностных алгоритмов (цепи Маркова), работа с n-граммами.
- **Структуры данных:** Эффективное использование словарей (хеш-таблиц), списков и массивов для хранения состояний и переходов.
- **Обработка данных:** Очистка текста, токенизация, работа с файловым вводом-выводом, генерация синтетических данных для обучения.
- **Основы UI:** Создание простого пользовательского интерфейса (веб-интерфейс, GUI или CLI) для взаимодействия с алгоритмом.
- **Работа с инструментами разработки:** Использование экосистемы Сфера (Сфера.Код, Сфера.Задачи) для ведения проекта.

Универсальные компетенции (soft skills):

- **Командная работа:** Распределение ролей (кто пишет алгоритм, кто интерфейс, кто готовит данные).
- **Планирование:** Декомпозиция задачи на спринты, оценка трудоемкости.
- **Тестирование и отладка:** Поиск граничных случаев (пустой текст, уникальные слова), обеспечение стабильности работы.

3. Постановка задания

Название кейса: Проведение анализа защищенности приложения с помощью LLM моделей.

Цель выполнения кейса:

Разработать обвязку вокруг ИИ агента для решения конкретной задачи – проведения анализа защищенности приложения, используя набор инструментов ИБ, техник, ИИ инструменты, а также составленный план тестирования.

Конкретный образ результата:

Подготовленная среда (ИИ агент, настроенное подключение внешней большой языковой модели, инструменты ИБ, план работы) для проведения анализа защищенности приложения, развернутая локально на компьютере, которая позволяет:

1. Проводить первичную аналитику веб приложения;
2. Составлять план анализа защищенности, используемых инструментов;
3. Используя описанные выше инструменты, находить заложенные уязвимости;
4. Для каждого результата описывать проблему в заданном формате
5. Полученные результаты ранжировать с учетом критичности

Промежуточные показы (Milestones):

- **Показ 1 (Конец 3-й недели):** Изучение предметной области, знакомство с основными уязвимостями, характерными для веб приложений.
- **Показ 2 (Конец 6-й недели):** Знакомство с инструментами анализа защищенности, работа с ними как напрямую, так и через окно ИИ агента.
- **Показ 3 (Конец 9-й недели):** Выработка плана тестирования приложения с использованием изученных инструментов ИБ.
- **Показ 4 (Конец 12-й недели):** Работа с полученными результатами, оформление результатов под требуемый формат, их ранжирование.

Форма и формат отчетности (финальная):

1. **Работающий ИИ агент** (описание связки агента (Dockerfile), описание основных команд для установки и подготовки инструментов ИБ, план работы для агента, MCP сервера (опционально).
2. **Демонстрация полученных результатов на тренировочном тестовом стенде:** выявленные уязвимости, описанные в заданном формате.
3. **Проверка работы агента на проверочном тестовом стенде:** выявленные уязвимости, описанные в заданном формате.
4. **Финальная презентация** (10 слайдов).

Сроки выполнения: 15 недель.

4. Рекомендуемые инструменты

Task-трекер Сфера.Задачи для настройки доски, постановки задач с ролями по спринтам и фиксации дефектов.

Wiki-система Сфера.Знания для формирования и хранения технической документации.

Git-репозиторий Сфера.Код для создания, версионирования и хранения кода.

5. Методические рекомендации обучающимся

Рекомендуемый состав группы (3-4 человека):

- **Team Lead / Аналитик:** Организация работы в Сфера.Задачи, изучение предметной области, определение этапности работ, общий контроль за качеством решения, описание сценариев использования.

- **Исследователь направления SAST (статического анализа кода):**

Проработка подходов к проверке исходного кода, интерпретация полученных результатов, адаптация под заданный формат

- **Исследователь направления DAST (динамического анализа кода):**

Проработка подходов к проверке веб приложения на уровне сетевого взаимодействия с тренировочным тестовым стендом, интерпретация полученных результатов, адаптация под заданный формат.

- **Разработчик ИИ платформы:** подготовка инструментария для автоматизированной проверки выработанных гипотез.

Примерный план работы по спринтам (3 недели на спринт):

Спринт 1 (Недели 1-3): Исследование предметной области и проектирование архитектуры

- **Задачи:**

- **Аналитик/Team Lead:**

Изучение методологий оценки защищенности веб-приложений (OWASP Top 10, OWASP WSTG). Определение требований к формату вывода результатов (JSON/Markdown структура для каждой уязвимости: описание, критичность, рекомендации).

- **Разработчик ИИ платформы:**

Выбор технологического стека для создания агента на базе OpenCode (настройки, плагины), выбор базовой LLM-модели (локальной или облачной с API), проектирование структуры Dockerfile и окружения.

- **Исследователи SAST/DAST:**

Сбор и систематизация списка популярных открытых инструментов статического (SAST, например, Semgrep) и динамического (DAST, например, OWASP ZAP, Nuclei) анализа. Формирование списка гипотез: какие типы уязвимостей какие инструменты закрывают лучше всего.

- **Результат:**

- Документ с описанием предметной области и перечнем целевых уязвимостей.
 - Утвержденный формат «Отчета об уязвимости» (шаблон).
 - Архитектурная схема взаимодействия: Агент -> LLM -> Инструменты ИБ -> Анализ результата -> LLM (ранжирование).
 - Готовый базовый Docker-контейнер с установленной ОС, Python-средой и базовыми инструментами.

Спринт 2 (Недели 4-6): Интеграция инструментов ИБ и прототипирование взаимодействия

- **Задачи:**
 - **Аналитик/Team Lead:**
Описание сценариев использования агента для формирования первоначального плана аудита
 - **Разработчик ИИ платформы:**
Реализация функций (tools/actions) для вызова внешних утилит (SAST/DAST инструментов) из агента. Настройка передачи входных данных (код или URL) в инструменты и парсинг их сырых логов/отчетов.
 - **Исследователи SAST:**
Тестирование SAST-инструментов с уязвимостями на основе кода тренировочного тестового стенда. Настройка правил сканирования, чтобы минимизировать ложные срабатывания на этапе парсинга.
 - **Исследователи DAST:**
Настройка DAST-инструментов для автоматического сканирования на основе тренировочного тестового стенда. Настройка правил сканирования, чтобы минимизировать ложные срабатывания на этапе парсинга.
- **Результат:**
 - Работающий прототип агента, который умеет запускать SAST/DAST инструменты и получать их «сырые» результаты.
 - Настроенное взаимодействие с тренировочным стендом
 - Настроенное взаимодействие с кодом сервиса для SAST-анализа.
 - Подтвержденная работоспособность связки: Запуск сканирования -> Получение лога -> Корректная обработка в коде.

Спринт 3 (Недели 7-9): Реализация логики анализа, планирования и LLM-интерпретации

- **Задачи:**
 - **Аналитик/Team Lead:**
Разработка матрицы критичности уязвимостей для системы ранжирования. Описание логики, по которой LLM должна присваивать приоритет (например, RCE > SQLi > XSS > Info Disclosure).
 - **Разработчик ИИ платформы:**
Настройка промптов (Prompt Engineering) для LLM. Внедрение этапа, где LLM анализирует «сырые» результаты сканирования, фильтрует ложные срабатывания и переводит их в заданный формат. Реализация функции работы с Планом анализа: LLM на основе целей пользователя выбирает нужные инструменты из подключенных.
 - **Исследователи SAST/DAST:**
Калибровка параметров инструментов. Например, DAST может генерировать много шума при первом сканировании — настройка глубины сканирования. SAST требует точных правил — донастройка профилей.
- **Результат:**
Полнофункциональный агент, способный:
 - Принять вводные от пользователя.
 - Запустить выбранные инструменты.

- Проанализировать результаты с помощью LLM.
- Сформировать итоговый структурированный отчет.
- Реализованный механизм ранжирования результатов по критичности.

Спринт 4 (Недели 10-12): Тестирование на тренировочном стенде и отладка

- **Задачи:**

- **Аналитик/Team Lead:**

Документирование выявленных ограничений агента и способов их обхода.

- **Разработчик ИИ платформы:**

Улучшение промптов на основе ошибок. Если LLM неправильно классифицирует уязвимость или забывает формат — исправление инструкций и, возможно, добавление Few-shot examples (примеров) в промпт. Оптимизация скорости работы (кэширование ответов LLM, параллельные вызовы инструментов).

- **Исследователи SAST/DAST:**

Сравнение результатов работы агента с эталонными данными (что мы знаем об уязвимостях стенда). Выявление пропущенных уязвимостей (False Negatives) и ложных срабатываний (False Positives)

- **Результат:**

- Задokumentированные результаты анализа на тренировочном стенде.
 - Список исправлений и оптимизаций, внесенных в агент.
 - Стабильная версия агента, выдающая результаты в заданном формате.

Спринт 5 (Недели 13-15): Финальное тестирование, оформление и подготовка презентации

- **Задачи:**

- **Вся команда**

Запуск анализа на проверочном тестовом стенде (новый, неиспользованный ранее уязвимый веб-приложение)

- **Аналитик/Team Lead:**

Создание финальной презентации (10 слайдов): проблема, решение, архитектура агента, результаты тестов (сравнение стендов), выводы, демо-видео или скриншоты. Подготовка защитной речи.

- **Разработчик ИИ платформы:**

Подготовка финального Dockerfile, написание README.md с инструкциями по установке, запуску и использованию агента. Упаковка решения.

- **Исследователи SAST/DAST:**

Финальная проверка качества описания найденных уязвимостей. Убедиться, что рекомендации от LLM осмысленны и полезны для разработчика.

- **Результат:**

- Работающий ИИ агент, готовый к развертыванию локально (Dockerfile, документация).
 - Полный пакет отчетов по тестированию на двух стендах в требуемом формате.
 - Готовая презентация и скомпилированный финальный отчет.

6. Методические рекомендации по оцениванию

Общая система оценивания:

Максимум 100 баллов.

- 80–100: Отлично
- 60–79: Хорошо
- 40–59: Удовлетворительно
- 0–39: Неудовлетворительно

Критерии оценивания:

Блок 1. Промежуточные показы (40 баллов)

- **Показ 1 (10 б):** Качество подобранных данных (текстов), понимание задачи, настройка среды (Сфера).
- **Показ 2 (10 б):** Корректность парсинга текста. Эффективность выбранной структуры данных.
- **Показ 3 (10 б):** Качество генерации. Текст не должен быть точной копией исходника, но должен сохранять структуру. Отсутствие вылетов программы.
- **Показ 4 (10 б):** Наличие работающего интерфейса. Удобство использования.

Блок 2. Финальный продукт (40 баллов)

- **Алгоритмическая реализация (20 б):** Код чистый, переменные названы понятно. Алгоритм работает быстро даже на больших текстах.
- **Качество продукта (10 б):** Приложение не падает при некорректных данных (например, пустой файл). Интерфейс отзывчивый.
- **Функциональность (10 б):** Реализованы все заявленные функции (загрузка, настройка, генерация).

Блок 3. Документация и защита (20 баллов)

- **Документация (5 б):** Понятная инструкция, как запустить проект.
- **Презентация (10 б):** Демонстрация "живого" примера генерации. Внятные ответы на вопросы.
- **Понимание темы (5 б):** Студенты понимают, что такое цепь Маркова, и могут объяснить, как вероятность влияет на выбор следующего слова.

Описание уровней оценки (адаптировано под 2 семестр):

- **Отлично:** Приложение имеет приятный UI, позволяет настраивать "глубину" анализа (1 слово, 2 слова). Код хорошо структурирован. Студенты использовали систему контроля версий (Сфера.Код) регулярно.
- **Хорошо:** Приложение работает, текст генерируется. Интерфейс минималистичный (или консольный, но удобный). Есть мелкие баги в пунктуации.
- **Удовлетворительно:** Программа работает только на идеальных данных. Интерфейс неудобный или отсутствует (только запуск скрипта). Код написан "спагетти-стилем".
- **Неудовлетворительно:** Проект не собирается, генерация не работает или выдает бессмысленный набор символов (не слов). Нет командной работы.